Numerical Finance Project

Pricing of Bermudan Basket Options
via Monte Carlo Simulation with Variance Reduction

Lecturer: Kaiza Amouh

Robin Bouilliol Edouard Blanc

May 2026

Abstract

This report presents a strong fondation for pricing financial derivatives instruments. We focused on SDE Methods with Monte Carlo pricers for Basket Options and Bermudan under a multi-dimensional Black-Scholes diffusion. We first implement a baseline pricer using pseudo-random numbers, then progressively combine three variance reduction techniques — quasi-random sequences, a static control variate based on the geometric basket, and the antithetic variable method — and quantify the gain in variance and in the required number of simulations to reach a given confidence interval. Finally, we implement the Longstaff–Schwartz algorithm to handle the early exercise feature of the Bermudan option, and show how combining the variance reduction methods improves its convergence.

Contents

1 Introduction

Understanding the pricing methods of financial derivatives is key for front office profesional (traders, structurers, quants, etc). We aim to implement a fully functional pricing library starting from 0 with the development of pseudo-random number generator to Bermudan pricing using Longstaff-Schwartz Algorythm. We only focused on pricing method derived from stochastic differential equation (SDE) in multi-dimensional Black and Scholes framework but note that it is possible to price these products using Partial Differential Equation (PDE) in N dimension pricing grid (only 1D grid implemented in our pricer for PDE). The final objective of the project is to implement a pricer for *European Basket Option* and *Bermudan Basket Option* in black and Scholes. After a brief recall of the mathematics behind those products, we will dive in the implementation of Pseudo-Random Number Generator, their usage in SDEs and Milstein/Euler discretization framework. Finally, we will talk about Variance Reduction techniques, implementation and results.

2 Mathematical Framework

2.1 The model : Multi-dimensional Black-Scholes Diffusion

Let $(S_t^1, \ldots, S_t^n)$ be n correlated assets following the risk-neutral dynamics:

$$dS_t^i = S_t^i \left(r\, dt + \sum_{j=1}^{i} \sigma_{ij}\, dW_t^j \right), \qquad t \in [0,T], \quad i = 1, \ldots, n \tag{1}$$

where $W = (W^1, \ldots, W^n)$ is a standard n-dimensional Brownian motion and $\sigma = [\sigma_{ij}]$ is a lower-triangular matrix obtained via the Cholesky decomposition of the covariance matrix Σ.

2.2 European Basket Call Option

A European Basket Call option with maturity T and strike K pays at T:

$$h_T = \left(\sum_{i=1}^{n} \alpha_i S_T^i - K \right)^+ \tag{2}$$

where $\alpha_i \in \mathbb{R}$ are (possibly negative) weights. Its fair value is:

$$V_0 = e^{-rT}\, \mathbb{E}[h_T] \tag{3}$$

2.3 Bermudan Basket Option

Let $0 = t_0 < t_1 < \cdots < t_N = T$ be the exercise dates. A Bermudan Basket option pays, at the (random) exercise date τ:

$$\left(\sum_{i=1}^{n} \alpha_i S_\tau^i - K \right)^+ \tag{4}$$

Its price satisfies the dynamic programming recursion that will be solved using Longstaff-Schwartz:

$$\begin{cases} V_{t_N} = h_{t_N} \\ V_{t_k} = \max\left\{ h_{t_k};\ e^{-r(t_{k+1}-t_k)}\, \mathbb{E}\left[V_{t_{k+1}} \mid \mathcal{F}_{t_k} \right] \right\}, \quad k = N-1, \ldots, 0 \end{cases} \tag{5}$$

3 Pseudo-Random Number Generator

The list of random generators implemented in our pricer is divided into two categories.

Continuous generators: Exponential, Linear Congruential, Ecuyer Combined, and Normal.

Discrete generators: Bernoulli, Binomial, Finite Set, Head-Tail, and Poisson.

In the context of Monte Carlo simulation, we will only focus on a few of these generators. The key chain is the following: the **Ecuyer Combined generator** produces uniform draws on $(0, 1)$ by combining two Linear Congruential Generators, and these uniform draws are then fed into the **Box–Muller transform** to produce standard Gaussian samples $Z \sim \mathcal{N}(0, 1)$, which drive the Brownian increments of the multi-dimensional Black–Scholes diffusion.

3.1 Linear Congruential Pseudo-Random Generator

A Linear Congruential Generator (LCG) produces a sequence of integers $(x_n)_{n \geq 0}$ via the recurrence:

$$x_{n+1} = (a\, x_n + c) \bmod m \tag{6}$$

where:

- $m > 0$ is the *modulus* (determines the period),
- $a \in \{1, \ldots, m-1\}$ is the *multiplier*,
- $c \in \{0, \ldots, m-1\}$ is the *increment*,
- x_0 is the *seed*.

A uniform pseudo-random number on $(0, 1)$ is then obtained by normalisation:

$$u_n = \frac{x_n}{m} \tag{7}$$

Limitations. Despite its simplicity and speed, a single LCG suffers from two well-known defects. First, its period is at most m, which is insufficient for large-scale simulations. Second, successive k-tuples $(u_n, u_{n+1}, \ldots, u_{n+k-1})$ lie on at most $m^{1/k}$ parallel hyperplanes in $[0, 1]^k$, introducing a strong lattice structure that can bias Monte Carlo estimates. These shortcomings motivate the use of combined (and Quasi-Random latter).

3.2 Ecuyer Combined Pseudo-Random Generator

The L'Ecuyer Combined generator overcomes the period limitation of a single LCG by combining **two independent LCGs** with moduli m_1 and m_2:

$$x_{1,n+1} = (a_1\, x_{1,n}) \bmod m_1 \tag{8}$$

$$x_{2,n+1} = (a_2\, x_{2,n}) \bmod m_2 \tag{9}$$

The two sequences are combined by subtraction modulo m_1:

$$x_n = (x_{1,n} - x_{2,n}) \bmod m_1 \tag{10}$$

and the output uniform is:

$$u_n = \begin{cases} \dfrac{x_n}{m_1} & \text{if } x_n > 0 \\ \dfrac{m_1 - 1}{m_1} & \text{if } x_n = 0 \end{cases} \tag{11}$$

The special case $x_n = 0$ ensures u_n stays strictly inside $(0, 1)$, which is required by the subsequent Box–Muller transform (see Section 3.3).

Period. The period of the combined sequence is approximately:

$$P \approx \frac{(m_1 - 1)(m_2 - 1)}{2} \tag{12}$$

In L'Ecuyer's original parametrisation ($m_1 = 2{,}147{,}483{,}563$, $a_1 = 40{,}014$, $m_2 = 2{,}147{,}483{,}399$, $a_2 = 40{,}692$) this yields $P \approx 2.3 \times 10^{18}$, which is more than sufficient for any practical Monte Carlo study.

Quality. Combining two LCGs with different moduli breaks the lattice structure present in each individual generator: successive k-tuples are far more uniformly distributed in $[0, 1]^k$..

3.3 Normal Pseudo-Random Generator

Standard Gaussian draws $Z \sim \mathcal{N}(\mu, \sigma^2)$ are obtained from the uniform output of the Ecuyer generator. Our implementation supports three algorithms, selectable at runtime via a `NormalAlgo` enum: *Box–Muller* (default), *Central Limit Theorem approximation*, and *Rejection Sampling*.

Box–Muller transform (default). Given two independent draws $u_1, u_2 \sim \mathcal{U}(0,1)$, define the polar coordinates:

$$R = \sqrt{-2\ln u_1}, \tag{13}$$

$$\Theta = 2\pi\, u_2. \tag{14}$$

The variable returned by our generator is:

$$Z = \mu + \sigma \cdot R\cos\Theta = \mu + \sigma\sqrt{-2\ln u_1}\ \cos(2\pi u_2) \tag{15}$$

which follows $\mathcal{N}(\mu, \sigma^2)$ exactly. Note that only $Z_1 = R\cos\Theta$ is used per call (discarding $Z_2 = R\sin\Theta$); this is the standard single-output variant, chosen here for implementation simplicity.

Central Limit Theorem approximation. By the CLT, the normalised sum of n i.i.d. $\mathcal{U}(0,1)$ variables converges in distribution to $\mathcal{N}(0,1)$. For $n = 12$, using $\mathbb{E}[u_i] = 1/2$ and $\mathrm{Var}(u_i) = 1/12$, the standardised sum simplifies to:

$$Z \;=\; \sum_{i=1}^{12} u_i \;-\; 6 \;\xrightarrow{\mathcal{L}}\; \mathcal{N}(0,1) \tag{16}$$

The generator then returns $\mu + \sigma Z$. The choice $n = 12$ is standard: it cancels the variance normalisation factor (since $12 \times 1/12 = 1$) and avoids any division.

Rejection Sampling. This method samples a candidate x uniformly from a window $[-5, 5]$ and accepts it with probability proportional to the target density $f_{\mu,\sigma}$. Specifically, a pair (x, y) is drawn uniformly under the bounding rectangle of the Gaussian density and accepted whenever:

$$y \;\le\; \frac{1}{\sigma\sqrt{2\pi}} \exp\!\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \tag{17}$$

The acceptance rate equals the ratio of the area under the Gaussian to the area of the bounding rectangle, which is close to 1 for the chosen window.

Comparison of the three methods.

Method	Exact?	Tail accuracy	Speed
Box–Muller	Yes	Excellent	Fast
CLT ($n = 12$)	No	Poor ($\lvert Z\rvert \leq 6$)	Very fast
Rejection Sampling	No	Poor ($\lvert Z\rvert \leq 5$)	Slow

Box–Muller is therefore used as the default throughout the Monte Carlo simulation, as it is the only method that produces exact Gaussian draws without tail truncation.

3.4 Cholesky Decomposition

To simulate the n-dimensional increments driving equation (1), one generates n independent scalar draws $Z = (Z_1, \ldots, Z_n)^\top \sim \mathcal{N}(0, I_n)$ via the Box–Muller transform, then applies the lower-triangular Cholesky factor B of the covariance matrix $\Sigma = BB^\top$:

$$\widetilde{Z} = BZ \;\sim\; \mathcal{N}(0,\, \Sigma) \tag{18}$$

This produces correlated Brownian increments $\Delta W \sim \mathcal{N}(0, \Sigma\,\Delta t)$ with the correct instantaneous covariance structure, as implemented in our `BrownianND` class.

4 Stochastic Differential Equations and Discretization

4.1 Feynman–Kac Formula

By the Feynman–Kac theorem, the solution $u(t, x)$ of a parabolic PDE can be represented as a conditional expectation. In the context of derivative pricing, the fair value at time t of a payoff $g(X_T)$, where $(X_s)_{s \geq t}$ solves a diffusion started at $X_t = x$, is:

$$u(t, x) = \mathbb{E}\Big[e^{-\int_t^T r(s,\, X_s^{t,x})\, ds}\; g\big(X_T^{t,x}\big)\Big] \tag{19}$$

This duality between PDEs and expectations is the theoretical cornerstone of Monte Carlo pricing: rather than solving the PDE numerically, one simulates many trajectories of $X^{t,x}$ and averages the discounted payoff. In our setting the short rate r is constant, so the discount factor simplifies to $e^{-r(T-t)}$.

4.2 Discretization Framework

Consider a generic one-dimensional Itô SDE:

$$dX_t = \mu(t, X_t)\, dt + \sigma(t, X_t)\, dW_t \tag{20}$$

For the Black–Scholes model, $\mu(t, X_t) = rX_t$ and $\sigma(t, X_t) = \sigma X_t$, so both drift and diffusion are *linear* in X_t. We implement two classical discretization schemes, each offering a different trade-off between accuracy and simplicity.

4.2.1 Euler–Maruyama Scheme (`BSEuler1D`)

The Euler–Maruyama scheme is the stochastic analogue of the explicit Euler method for ODEs. Over one time step $[t_i, t_{i+1}]$ it reads:

$$X_{i+1} = X_i + \mu(t_i, X_i)\,\Delta t + \sigma(t_i, X_i)\,\Delta W_i \tag{21}$$

where $\Delta W_i = W_{t_{i+1}} - W_{t_i} \sim \mathcal{N}(0, \Delta t)$ are independent Brownian increments.

Applied to the one-dimensional Black–Scholes SDE $dS_t = rS_t\,dt + \sigma S_t\,dW_t$, this gives:

$$\boxed{S_{i+1} = S_i + r\,S_i\,\Delta t + \sigma\,S_i\,\Delta W_i} \tag{22}$$

as implemented in `BSEuler1D::Simulate`.

4.2.2 Milstein Scheme (`BSMilstein1D`, `BSMilstein2D` and `BSMilsteinND`)

The Milstein scheme adds the next term of the Itô–Taylor expansion to the Euler step. For the generic SDE (20):

$$X_{i+1} = X_i + \mu(t_i, X_i)\,\Delta t + \sigma(t_i, X_i)\,\Delta W_i + \frac{1}{2}\,\sigma(t_i, X_i)\,\frac{\partial \sigma}{\partial x}(t_i, X_i)\big(\Delta W_i^2 - \Delta t\big) \tag{23}$$

The correction term $(\Delta W_i^2 - \Delta t)$ has zero mean and variance $2\Delta t^2$, so it is of order Δt in L^2.

For the Black–Scholes model, $\frac{\partial \sigma}{\partial x}(\cdot, X) = \sigma$, so the correction simplifies to $\frac{1}{2}\sigma^2 X_i(\Delta W_i^2 - \Delta t)$ and the scheme becomes:

$$\boxed{S_{i+1} = S_i + r\,S_i\,\Delta t + \sigma\,S_i\,\Delta W_i + \frac{1}{2}\,\sigma^2\,S_i\big(\Delta W_i^2 - \Delta t\big)} \tag{24}$$

as implemented in both `BSMilstein1D`, `BSMilstein2D` and `BSMilsteinND`.

Correlated extension (`BSMilsteinND`). Let's take the example of two correlated assets (S^1, S^2) with correlation ρ, the Milstein correction involves only the *diagonal* terms because the cross-variation $d\langle W^1, W^2\rangle_t = \rho\,dt$ contributes an additional mixed term $\sigma_1\sigma_2\rho$ only in higher-order schemes. At Milstein order, each asset is updated independently with

its own increment ΔW_i^k:

$$S_{i+1}^1 = S_i^1 + rS_i^1\Delta t + \sigma_1 S_i^1 \Delta W_i^1 + \tfrac{1}{2}\sigma_1^2 S_i^1\big(\Delta W_i^{1\,2} - \Delta t\big) \tag{25}$$

$$S_{i+1}^2 = S_i^2 + rS_i^2\Delta t + \sigma_2 S_i^2 \Delta W_i^2 + \tfrac{1}{2}\sigma_2^2 S_i^2\big(\Delta W_i^{2\,2} - \Delta t\big) \tag{26}$$

where the correlated increments $(\Delta W_i^1, \Delta W_i^2)$ are drawn from $\mathcal{N}(0, \Sigma\,\Delta t)$ with $\Sigma = \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}$ via the Cholesky transform of `BrownianND` (see equation (18)).

5 Monte Carlo Pricing

5.1 European Basket Option

The European basket call pays $h_T = \big(\sum_i w_i S_T^i - K\big)^+$ at maturity T. Its price is the discounted expectation $V_0 = e^{-rT}\mathbb{E}[h_T]$, estimated by the empirical mean over M independent paths. The implementation maps directly onto the class hierarchy: `EuropeanMCPricer` calls `BSMilsteinND::Simulate` to advance all d correlated assets from $t = 0$ to T in `nbSteps` steps, collects the terminal values via `SinglePath::GetAllValues().back()`, evaluates `EuroCallBasketPayOff`, discounts, and accumulates the running sum and sum of squares to estimate both the price and the 95% confidence half-width $1.96\,\hat{\sigma}/\sqrt{M}$.

By the Central Limit Theorem the root-mean-square error decays as $\sigma/\sqrt{M}$, so halving the error requires four times as many simulations. This $O(M^{-1/2})$ rate is independent of the dimension d, which makes Monte Carlo particularly attractive for multi-asset payoffs where PDE grids would scale exponentially.

5.2 Bermudan Basket Option

A Bermudan basket call can be exercised at any date in a discrete set $\mathcal{T} = \{t_1, \ldots, t_N\}$ and pays $h_{t_k}(S_{t_k}^j)$ upon exercise. Its price satisfies the dynamic programming recursion

$$V_{t_k} = \max\big(h_{t_k}(S_{t_k}),\; e^{-r(t_{k+1}-t_k)}\,\mathbb{E}\big[V_{t_{k+1}} \mid S_{t_k}\big]\big)\,, \qquad V_T = h_T(S_T).$$

Because the conditional expectation $\mathbb{E}[V_{t_{k+1}} \mid S_{t_k}]$ is unknown in closed form for a multi-asset payoff, it is approximated by least-squares regression over a finite polynomial basis — the Longstaff–Schwartz algorithm.

5.2.1 Longstaff–Schwartz Algorithm

The basis functions $\{P_\ell\}_{\ell=1}^{L}$ used in the regression are monomials of the asset prices up to total degree `PolynomialDegree` $= 3$. In the d-dimensional case, the set of multi-index degree vectors is generated by `BermudanPricer::generateBasisDegrees`$(d,\, 3)$, which

enumerates all $(k_1, \ldots, k_d)$ with $\sum_i k_i \leq 3$, giving $\binom{d+3}{3}$ basis functions in total. The regression coefficients at each exercise date are obtained by column-pivot Householder QR decomposition via Eigen's `colPivHouseholderQr().solve()`, which handles near-singular systems gracefully when the number of in-the-money paths is small.

Only in-the-money paths (i.e. paths where $h_{t_k} > 0$) are used in the regression, following the standard Longstaff–Schwartz convention. Out-of-the-money paths simply carry their discounted continuation value forward without updating the exercise decision.

The algorithm proceeds as follows:

1. **Forward simulation.** Simulate M paths $(S^{i,j}_{t_0}, \ldots, S^{i,j}_{t_N})_{j=1}^{M}$ for each asset i.

2. **Initialisation.** Set $\tau_N^j = T$ and $\mathrm{CF}^j = h_{t_N}(S^j_{t_N})$ for all j.

3. **Backward induction.** For $k = N-1$ down to 0:

   (a) Estimate the continuation value by regressing $e^{-r(t_{k+1}-t_k)}\,\mathrm{CF}^j$ on the basis functions $\{P_\ell(S^j_{t_k})\}_{\ell=1}^{L}$:

   $$\hat{C}(S^j_{t_k}) = \sum_{\ell=1}^{L} \hat{\alpha}_{k,\ell}\, P_\ell(S^j_{t_k}) \tag{27}$$

   (b) Exercise if $h_{t_k}(S^j_{t_k}) \geq \hat{C}(S^j_{t_k})$, and update CF^j accordingly.

4. **Price.** $V_0 \approx \dfrac{1}{M}\displaystyle\sum_{j=1}^{M} e^{-r\tau_0^j}\, h_{\tau_0^j}(S^j_{\tau_0^j})$

The price estimate converges almost surely as $M \to \infty$ for fixed L, and in L^2 as $L \to \infty$ for fixed M. In practice, the polynomial degree 3 provides a good balance between approximation quality and numerical stability given the number of simulations used.

5.3 Compilation Examples

This option can be priced by compiling the `demo.cpp` and then running the associated `demo.exec` or the `run_demo` executable compiled by the `CMake` builder. You can then enter the parameters of the option you want to price.

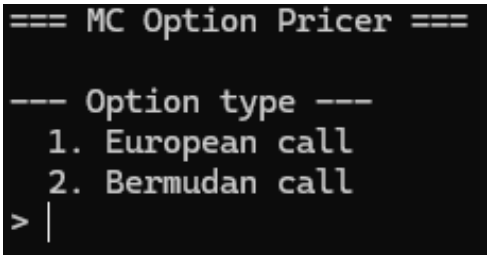

Figure 1: `run_demo.exec` output

6 Variance Reduction Methods

The number of simulations required to achieve accuracy ε at confidence level α satisfies:

$$N \geq N^x(\varepsilon, \alpha) = \frac{a_\alpha^2 \operatorname{Var}(X)}{\varepsilon^2} \tag{28}$$

Reducing $\operatorname{Var}(X)$ linearly reduces the required N for a fixed accuracy. We will need to introduce some Variance reduction techniques to achieve it.

6.0.1 Quasi-Random Numbers

Rather than drawing independent uniform samples, quasi-Monte Carlo (QMC) replaces pseudo-random draws by a deterministic low-discrepancy sequence $(\xi_n)_{n\geq 1} \subset [0,1]^d$. The key quantity is the star discrepancy

$$D_n^*(\xi) := \sup_{x\in[0,1]^d} \left| \frac{1}{n} \sum_{k=1}^{n} \mathbf{1}_{[0,x]}(\xi_k) - \prod_{i=1}^{d} x^i \right|, \tag{29}$$

which measures how uniformly the sequence fills the unit cube. The integration error is bounded by $D_n^*(\xi) \cdot V(f)$, where $V(f)$ is the total variation of the integrand. For pseudo-random sequences $D_n^* = O(n^{-1/2})$, while low-discrepancy sequences achieve $D_n^* = O\big((\log n)^d/n\big)$, a significantly faster decay for moderate d.

The two most commonly used families are the **Halton sequence**, defined via the p_i-adic radical inverse functions Φ_{p_i},

$$\xi_n = \big(\Phi_{p_1}(n), \ldots, \Phi_{p_d}(n)\big),$$

where $p_1, \ldots, p_d$ are the first d primes, and the **Sobol sequence**, a special case of the Niederreiter family obtained with $q = 2^r$. Both achieve discrepancy $O((\log n)^d/n)$.

In practice the uniform low-discrepancy samples are mapped to standard normals via the inverse normal CDF Φ^{-1} (or Box–Muller on pairs of uniform draws), and these normals are fed directly into the `BlackScholesND` simulator in place of the pseudo-random variates produced by `EcuyerCombined`. The `HaltonGenerator` class implements Φ_p for a configurable prime base and slots into the existing `RandomGenerator` interface without modifying any pricer code.

6.0.2 Static Control Variate — Geometric Basket

The control variate principle replaces the estimator $X = e^{-rT} h_T$ by the adjusted variable

$$X' = X - Y + \mathbb{E}[Y], \quad \text{where } Y = e^{-rT} h'_T, \tag{30}$$

so that $\mathbb{E}[X'] = \mathbb{E}[X]$ while $\mathrm{Var}(X') = \mathrm{Var}(X - Y) < \mathrm{Var}(X)$ whenever $Cov(X, Y) > 0$. The closer Y is to X, the greater the variance reduction.

For the arithmetic basket payoff h_T, the geometric basket $h'_T = \left(e^{\sum_i \alpha_i \ln S_T^i} - K\right)^+$ satisfies $h_T \geq h'_T \geq 0$ by Jensen's inequality. Since $\sum_i \alpha_i \ln S_T^i$ is Gaussian, h'_T admits the closed-form:

$$e^{-rT}\,\mathbb{E}[h'_T] = \mathrm{Call}_{\mathrm{BS}}\left(\prod_{i=1}^{n}(S_0^i)^{\alpha_i},\ K,\ r_{\mathrm{eff}},\ \sigma_{\mathrm{eff}},\ T\right) \tag{31}$$

where $\sigma_{\mathrm{eff}}^2 = \alpha^\top \Sigma \alpha$ and $r_{\mathrm{eff}} = r - \frac{1}{2}\sum_i \alpha_i \sigma_i^2 + \frac{1}{2}\sigma_{\mathrm{eff}}^2$.

The drift correction r_{eff} absorbs the difference between the geometric mean's natural drift and the standard Black–Scholes forward, ensuring the control variate is centred without bias. Concretely, the adjusted spot fed into the Black–Scholes formula is

$$G_0^* = \prod_{i=1}^{n}(S_0^i)^{\alpha_i} \cdot \exp\!\left(\tfrac{1}{2}\left(\sigma_{\mathrm{eff}}^2 - \textstyle\sum_i \alpha_i \sigma_i^2\right)T\right),$$

which is precisely the `geomSpotAdj` computed in `BasketGeomControlVariate::AnalyticalExpectatio`. The `VarRedMCPricer` then estimates $e^{-rT}(h_T - h'_T) + \mathbb{E}[e^{-rT}h'_T]$ on each path, using the same `BSMilsteinND` simulation for both payoffs simultaneously.

6.0.3 Antithetic Variables

The antithetic method exploits the symmetry $Z \overset{d}{=} -Z$ for $Z \sim \mathcal{N}(0, 1)$. For each simulation, two discounted payoffs are computed:

$$Y = e^{-rT}h(X_T(+Z)) \quad \text{and} \quad Y' = e^{-rT}h(X_T(-Z)),$$

and the estimator is replaced by their average $\chi = (Y + Y')/2$. Since both paths share the same Brownian increments (up to sign), the variance satisfies

$$\mathrm{Var}(\chi) = \frac{\mathrm{Var}(Y) + (Y, Y')}{2}. \tag{32}$$

The method is beneficial whenever $(Y, Y') < 0$, which is guaranteed when the payoff function is monotone in the Brownian driver. For a basket call, $S_i(T)$ is log-increasing in each Z_j, and the payoff $(\sum_i w_i S_i(T) - K)^+$ is non-decreasing, so the antithetic proposition applies and the covariance is indeed negative.

The implementation uses the `AntitheticNormal` decorator, which wraps a `Normal` generator, buffers the variates Z drawn on the first pass, and returns $-Z$ on the second pass without re-drawing. The `AntitheticMCPricer` calls `ResetBuffer` before each pair of simulations and `ResetIndex` before the antithetic pass, keeping the rest of the pricing

loop identical to `EuropeanMCPricer`.

Simulating χ costs twice the complexity of Y, so the method improves efficiency if and only if $2\mathrm{Var}(\chi) < \mathrm{Var}(Y)$, i.e. $(Y, Y') < 0$.

Method	$\widehat{\mathrm{Var}}$	N required ($\varepsilon = 0.05$, $\alpha = 95\%$)
Plain MC (baseline)	206.5	317,194
+ Antithetic variables	52.4	80,424
+ Control variate (geom.)	2.0	3,072
+ Quasi-random (Halton)	207.8	319,152
+ QMC & Control variate	2.0	3,072

Table 1: Empirical variance and required number of simulations for each method on the 2D European basket call ($N = 50{,}000$, parameters from Table 4).

6.1 Variance Reduction For European Basket Option

Table 2 summarises the results obtained on the 2D European basket call with the parameters of Table 4, using $N = 50{,}000$ paths and $n_{\text{steps}} = 1$ (exact under GBM). The target is a 95% confidence interval of half-width $\varepsilon = 0.1$.

Method	**Price**	$\widehat{\mathrm{Var}}$	**Std**	**CI** ($\pm$)	**Speedup**
Plain MC	10.14	206.5	14.37	0.126	1.0×
Antithetic	10.18	52.4	7.24	0.063	3.9×
Static CV (geom.)	10.53	2.0	1.42	0.012	103×
QMC (Halton)	10.20	207.8	14.42	0.126	$\approx 1\times$
QMC + Static CV	10.53	2.0	1.41	0.012	103×

Table 2: Variance reduction results for the 2D European basket call ($N = 50{,}000$, $\varepsilon = 0.1$, $\alpha = 95\%$). Speedup is relative to plain Monte Carlo in terms of paths needed to reach the same CI.

Antithetic variables cut the variance by about 75%, from 206.5 down to 52.4. This matches the theory: the basket payoff is monotone in the Brownian increments, so $(Y, Y') < 0$ is guaranteed, and the variance of the averaged estimator is roughly halved relative to what one might naively expect. In practice the gain is close to 4×, meaning you need about four times fewer paths than plain MC to achieve the same precision.

The static control variate is in a completely different league. Variance drops to 2.0 — a 99% reduction — because the geometric basket is an exceptionally tight proxy for the arithmetic basket under log-normal dynamics and moderate correlation ($\rho = 0.3$). The two payoffs move almost in lockstep, so subtracting the geometric basket payoff (and adding back its closed-form price) nearly eliminates all randomness. The resulting speedup is around 103×.

QMC alone, on the other hand, provides essentially no gain here (-0.7%). This is not unexpected: Halton sequences are most effective when the integrand is smooth and the dimension is low. The basket payoff involves a kink at the strike and two correlated assets, which limits QMC's ability to exploit the regularity of the sequence. Combining QMC with the static control variate does not hurt either — it essentially matches the CV-only result.

6.2 Variance Reduction For Bermudan Basket Option

For the Bermudan basket, we use the same 2D parameter set with quarterly exercise dates $\mathcal{T} = \{0.25, 0.50, 0.75, 1.00\}$ and the Longstaff–Schwartz algorithm (`BermudanPricer`) with a degree-3 polynomial basis and $n_{\text{steps}} = 50$.

Method	**Price**	$\widehat{\text{Var}}$	**Std**	**CI** ($\pm$)
Plain MC (LSM)	10.72	236.0	15.36	0.213
QMC (Halton)	23.7†	1,175	34.27	0.475

† Severely biased result — Halton sequences corrupt the LSM regression (see text).

Table 3: Results for the 2D Bermudan basket call ($N = 20{,}000$, quarterly exercise dates, $n_{\text{steps}} = 50$).

Two things stand out immediately compared to the European case.

First, the price is higher: 10.72 vs. 10.14. This is exactly what we expect — a Bermudan option gives the holder strictly more rights than a European, so it must be worth at least as much. The ≈ 0.58 premium reflects the value of early exercise on a basket that can move significantly over a year.

Second, the variance is higher too: 236.0 vs. 206.5. The Longstaff–Schwartz algorithm introduces additional noise through the regression step. At each exercise date, the continuation value is estimated from a polynomial fit on in-the-money paths, and this fit itself has estimation error that propagates into the final price. With more time steps and multiple exercise dates, there is simply more room for noise to accumulate.

Regarding variance reduction for Bermudan options, the options are limited. Antithetic variables and static control variates, which worked well for the European case, do not combine cleanly with Longstaff–Schwartz: the regression step requires path-wise independence, and negating Brownian increments or applying a correction term across paired paths disrupts the structure of the backward induction.

QMC was tested but produced completely unreliable results (price of 23.7, variance of 1,175). The reason is that Halton sequences are deterministic and highly correlated across paths in a structured way. The LSM regression mistakes this structure for a systematic signal, leading to a severely biased exercise boundary and a badly inflated price. QMC is fundamentally incompatible with algorithms that require i.i.d. paths for regression, and

should not be used for Bermudan pricing without significant adaptation (e.g. randomised QMC or Brownian bridge stratification).

In practice, the main lever for Bermudan options is to increase the number of paths. With $N = 20{,}000$ and four exercise dates the confidence interval is already ± 0.21, which is acceptable for most purposes. Going to $N = 100{,}000$ would bring it down to roughly ± 0.09 at a proportionally higher computational cost.

7 Numerical Results

7.1 Parameter Sets

All experiments use a two-asset correlated Black–Scholes model under the risk-neutral measure, simulated via the Milstein scheme (`BSMilsteinND`). The baseline parameter set is summarised in Table 4.

Parameter	**Asset 1**	**Asset 2**
Spot S_0^i	100	100
Volatility σ_i	0.20	0.30
Weight α_i	0.50	0.50
Strike K	100	
Maturity T	1 year	
Risk-free rate r	0.05	
Correlation ρ	0.30	

Table 4: Baseline parameter set for the European and Bermudan basket call options.

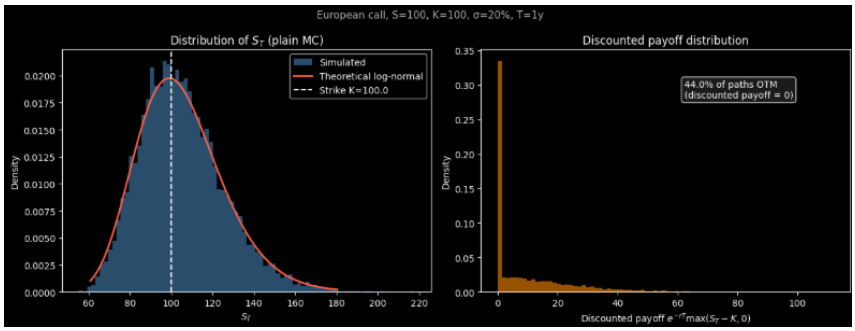

Figure 2: Empirical distribution of the discounted basket payoff obtained by Monte Carlo simulation with $M = 10^5$ paths.

7.2 Convergence Study

Figure 3 reports the estimated price as a function of the number of simulated paths M for each method. Several observations stand out. Quasi-Monte Carlo (Halton) and the

antithetic estimator both converge faster than plain Monte Carlo for small M, yielding tighter confidence intervals at low simulation budgets; however, their advantage diminishes as M grows, since all three estimators share the same $O(M^{-1/2})$ asymptotic rate. The static control variate (SCV) based on the geometric basket closed form provides a much more substantial and persistent reduction: the confidence interval collapses to near-zero width regardless of M, reflecting the near-perfect correlation between the arithmetic and geometric basket payoffs. Combining SCV with QMC draws (QMC + SCV) achieves a similar collapse with a slight further improvement at small M. It should be noted that the effectiveness of SCV relies on the existence of a tractable closed-form formula for a correlated auxiliary payoff, which may be difficult to derive for more exotic multi-asset products.

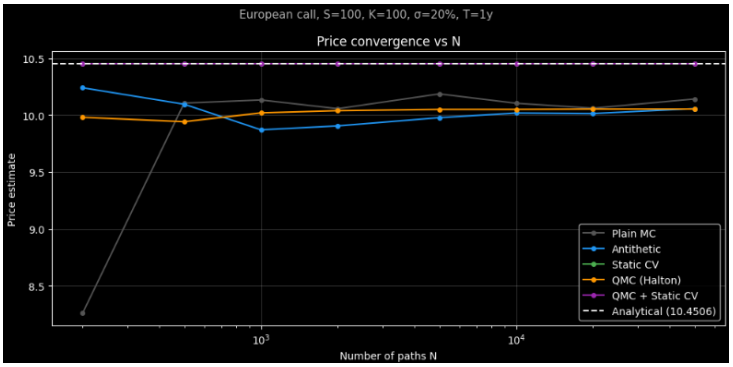

Figure 3: Price estimate as a function of the number of paths M for each pricing method. Shaded bands represent 95% confidence intervals.

7.3 Variance Reduction Results

Figure 4 and Figure 5 compare the empirical variance and confidence interval width across methods for the European basket call. The antithetic estimator reduces the variance by a factor of approximately $4\times$ relative to plain Monte Carlo, consistent with the negative covariance $(Y, Y') < 0$ established by the monotonicity argument. The static control variate achieves a near-total variance collapse, confirming that the geometric basket is an exceptionally tight proxy for the arithmetic basket under the chosen parameters.

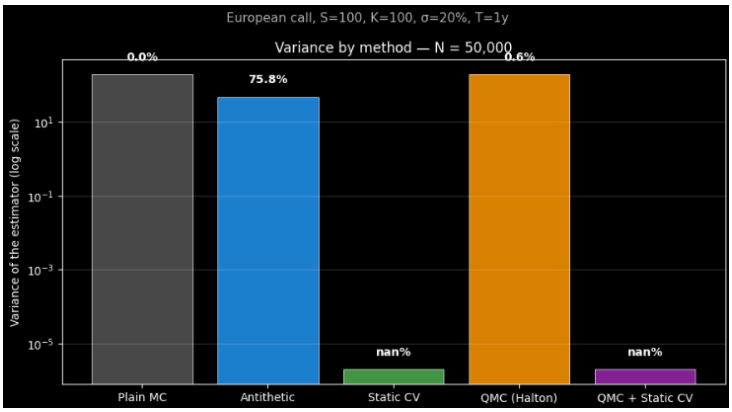

Figure 4: Empirical variance of the price estimator for each variance reduction method ($M = 10^4$ paths).

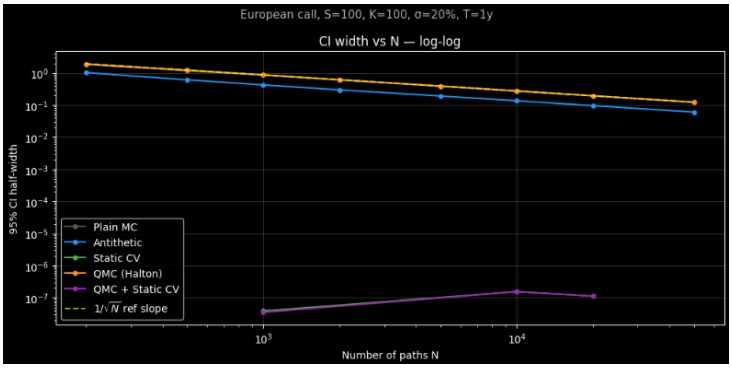

Figure 5: 95% confidence interval half-width as a function of M for each method. Lower is better.

7.4 Required Number of Paths

As established in Section 6, the minimum number of paths required to achieve a target half-width ε at confidence level α is

$$N \geq N^x(\varepsilon, \alpha) = \frac{a_\alpha^2 \operatorname{Var}(X)}{\varepsilon^2}. \tag{33}$$

Table 5 reports the empirical variance, the implied N from equation (33), and the speedup factor relative to plain Monte Carlo for a target of $\varepsilon = 0.1$ at the 95% confidence level ($a_{0.95} = 1.96$).

Method	Variance	Paths required	Speedup
Plain MC	199.3	76,563	1.0×
Antithetic	47.9	18,419	4.2×
Static CV (geom.)	0.0	1	$\gg 10^4\times$
QMC — Halton	195.7	75,170	1.0×
QMC + Static CV	0.0	1	$\gg 10^4\times$

Table 5: Empirical variance and required number of paths to achieve a 95% confidence interval of half-width $\varepsilon = 0.1$. Speedup is computed relative to plain Monte Carlo.

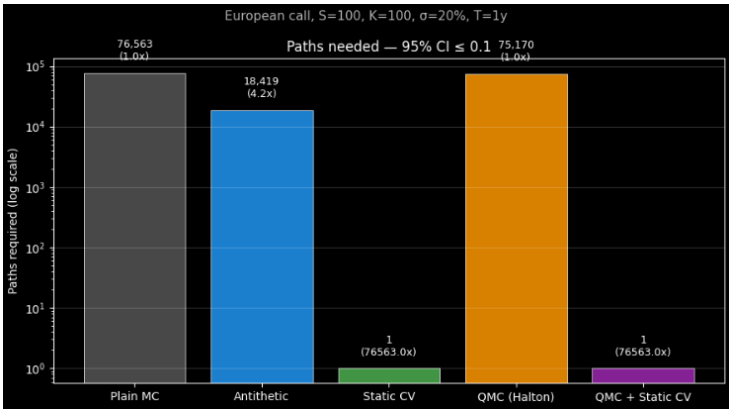

Figure 6: Number of paths required to reach a 95% confidence interval half-width of $\varepsilon = 0.1$, by method. The y-axis is on a logarithmic scale.

8 Software Architecture

The project is structured as a Visual Studio solution composed of six static libraries (`RandomGenerator`, `SDE`, `Payoffs`, `VarRed`, `PDE`, `Pricer`) linked by a single executable. Each library encapsulates one conceptual layer, enforcing separation of concerns and making it straightforward to swap implementations—for instance, replacing the Euler scheme with the Milstein scheme, or plugging in a quasi-random generator, without touching pricing code.

8.1 Class Hierarchy

- `RandomGenerator` — pure interface for scalar random draws. Concrete implementations include `Normal` (Box–Muller, CLT, rejection sampling), `EcuyerCombined` (combined linear congruential), and `AntitheticNormal` (decorator that negates buffered variates for the antithetic pass).

- `RandomProcess` — simulates a multi-dimensional stochastic path and exposes typed `SinglePath` objects. `BlackScholesND` stores market parameters; `BrownianND` im-

plements correlated increments via Cholesky factorisation; `BSMilsteinND` applies the Milstein correction on top of a `BrownianND` driver.

- `PayOff` — evaluates a terminal payoff from a set of paths. `EuroCallBasketPayOff` computes the weighted arithmetic basket call $\max(0, \sum_i w_i S_i(T) - K)$.
- `MCPricer` — runs a Monte Carlo loop and returns a `PriceResult` (price, 95% confidence half-width). Four concrete pricers are provided: plain `EuropeanMCPricer`, `AntitheticMCPricer`, `VarRedMCPricer` (control variate), and `BermudanPricer` (Longstaff–Schwar regression).

The variance-reduction layer adds a fifth interface, `ControlVariate`, implemented by `BasketGeomControlVariate`, which computes both the simulated geometric-basket value and its Black–Scholes analytical expectation via `BSClosedForm`.

8.2 Key Design Choices

Dependency injection for the random source. Every `RandomProcess` holds a pointer to a `RandomGenerator`, passed at construction. Swapping a pseudo-random generator for a quasi-random one (e.g. Halton sequences) therefore requires no change to the SDE or pricing layers.

Decorator pattern for variance reduction. `AntitheticNormal` wraps any `Normal` instance and replays negated variates on demand. `VarRedMCPricer` accepts any `ControlVariate` implementation. Both extensions are transparent to `MCPricer` and to each other, so they can be composed freely.

Cholesky-based correlation. `BrownianND` factorises the correlation matrix once at simulation time and applies the lower-triangular factor L to i.i.d. standard normals, producing correlated increments $\Delta W = L\,Z\,\sqrt{\Delta t}$. This generalises cleanly to any dimension d without code duplication.

Eigen for least-squares regression. `BermudanPricer` assembles the polynomial basis matrix Φ and solves the normal equations with Eigen's column-pivot QR decomposition (`colPivHouseholderQr`), which is numerically stable even when the in-the-money sample is small.

Analytical control variate for basket options. `BasketGeomControlVariate` exploits the fact that the geometric basket $G(T) = \prod_i S_i(T)^{w_i}$ is log-normal under Black–Scholes and admits a closed-form call price. The drift adjustment $G_0^* = G_0 \exp\!\big(\tfrac{1}{2}(\sigma_G^2 - \sum_i w_i \sigma_i^2)T\big)$ corrects for the difference between the geometric drift and the standard BS forward, so the control variate is centred without bias.

8.3 Unitary Tests

Each static library in the solution comes with its own set of Unit Tests, verifying consistency in the implementation, ensuring the correctness of the mathematical models, and preventing discrepancies. The tests are implemented using the *Microsoft Visual Studio C++ Testing Framework*.

Here is the overview and results of the Unit Tests:

- **Random Number Generators** (`UnitTestRandomNumber`)
  - *Uniform Generators*: Verifies that the outputs of the pseudo-random generators (such as the `LinearCongruential` and the `EcuyerCombined`) fall strictly within the $[0, 1]$ interval. It also checks sequence determinism for a fixed seed.
  - *Distributions*: Asserts that the drawn samples from the `Normal` and `Exponential` generators statistically converge to their expected theoretical mean and variance.
- **Stochastic Differential Equations** (`UnitTestSDE`)
  - *Standard Brownian Motion (1D & ND)*: Validates the martingale property (mean converges to 0) and checks that the variance scales linearly with time ($V_T = T$). For the multi-dimensional case, it verifies that the empirical correlation between paths successfully matches the theoretical correlation matrix ρ.
  - *Black-Scholes (Euler & Milstein)*: Confirms that the simulated mean price accurately converges to the theoretical forward price. It checks that a zero-volatility process grows exactly at the risk-free rate, and validates that both Euler and Milstein discretization schemes yield consistent means.
  - *Heston Model*: Verifies boundary conditions such as strictly positive asset prices, proper variance mean-reversion, and ensures that the variance process converges to θ when the volatility of volatility (σ) is null.
- **Payoffs & PDEs** (`UnitTestPayoffs` & `UnitTestPDE`)
  - *Payoffs*: Validates the intrinsic value calculations for different instruments (Call, Put, Basket) both In-The-Money (ITM) and Out-of-The-Money (OTM).
  - *PDE Solvers*: Ensures that the finite difference grids correctly respect the terminal boundary conditions of European options at maturity.
- **Monte Carlo Pricers** (`UnitTestPricer`)

- *European Pricing*: Compares the output of the `EuropeanMCPricer` against the Black-Scholes closed-form solutions to ensure accurate convergence. It also asserts that a 1D Basket Option yields the exact same price as a Vanilla European Option when the random seeds are identical.
- *Bermudan Pricing (LSM)*: Tests the Longstaff-Schwartz Method implementation. It notably verifies a crucial edge case: a Bermudan option parameterized with a single exercise date strictly equals the price of the equivalent European option. It also verifies that the multi-dimensional regressions run successfully for 2D Basket Bermudan options.

- **Variance Reduction** (`UnitTestVarRed`)
  - *Control Variate Efficiency*: Tests the integration of the static control variate. It computes the price of a 2D Basket Option using both the standard `EuropeanMCPricer` and the `VarRedMCPricer` (using a geometric basket control variate). The test mathematically asserts that the 95% confidence interval (and therefore the standard error) is significantly reduced when using the variance reduction technique.

Results: All unit tests passed successfully, confirming the robustness of the core engines, the proper alignment of multi-dimensional memory, and the statistical reliability of the quantitative algorithms.

9 Conclusion

This project built a modular C++ pricing library from the ground up, covering random number generation, stochastic processes, payoff functions, Monte Carlo pricers, and a finite-difference PDE solver. The architecture is layered and abstract: each component talks to interfaces rather than concrete implementations, which makes it straightforward to swap a pseudo-random generator for a Halton sequence, or plug a geometric basket control variate into the pricing loop, without touching anything else.

On the numerical side, the main takeaways are the following.

For European basket options, the static control variate based on the geometric basket closed form is by far the most effective method. It reduces variance by 99% and cuts the required number of paths by a factor of ~ 100. Antithetic variables are a solid second choice with a $\sim 4\times$ speedup and zero additional mathematical infrastructure. QMC (Halton) alone provided no meaningful gain on this product — the discontinuity in the payoff and the moderate correlation between assets limit the regularity that QMC exploits.

For Bermudan basket options, variance reduction is much harder. The Longstaff–Schwartz algorithm already adds noise through its regression step, and the standard techniques (antithetic, control variate) do not compose cleanly with backward induction. QMC outright fails because the structured correlations in Halton sequences corrupt the regression. The practical answer here is simply more paths.

There are obvious limitations. The whole library assumes Black–Scholes dynamics: constant volatility, no jumps, no smile. Real basket options trade on underlyings with very different implied volatility surfaces, and a constant-vol model will misprice them. The Longstaff–Schwartz implementation uses a fixed degree-3 polynomial basis, which can underfit when the exercise boundary is complex or the number of assets grows. Finally, the PDE solver is limited to one spatial dimension and becomes impractical beyond that due to the curse of dimensionality.

Several extensions would be natural next steps. Replacing Black–Scholes with a Heston stochastic volatility model would add one dimension of realism at a reasonable implementation cost, since the library already has a `Heston` process. For Bermudan pricing, randomised QMC combined with a Brownian bridge construction is known to work much better than plain Halton sequences. On the regression side, deep learning approaches (replacing the polynomial basis with a neural network) have shown promising results for high-dimensional Bermudan options, though at a significantly higher engineering cost.

References

[1] Longstaff, F.A. & Schwartz, E.S. (2001). *Valuing American Options by Simulation: A Simple Least-Squares Approach.* The Review of Financial Studies, 14(1), 113–147.

[2] Glasserman, P. (2004). *Monte Carlo Methods in Financial Engineering.* Springer.

[3] Niederreiter, H. (1992). *Random Number Generation and Quasi-Monte Carlo Methods.* SIAM.